DS1 MINI PROJECT · SPRING BOOT

CourseHub

Online course platform — REST API

Spring Boot 3 · Spring Data JPA · MySQL 8 · Postman

Mahmoud Najar

5-minute presentation · November 2026

01 · OVERVIEW

A complete backend for an online course platform

Instructors publish courses. Students enroll, track their progress and review them. Everything is exposed as a JSON REST API — there is no frontend, Postman is the client.

7
linked JPA entities

40+
REST endpoints, full CRUD

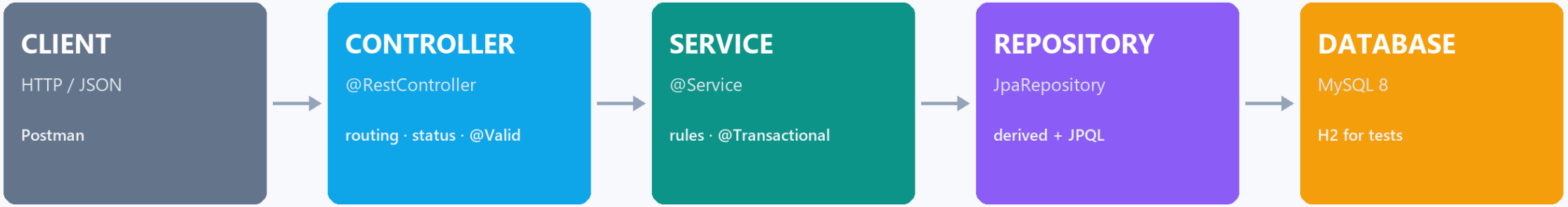
76
Postman checks, all passing

Category · Instructor · Tag · Course · Student · Enrollment · Review

One project, the whole DS1 checklist: entities, relationships, CRUD, custom queries, validation, error handling and Postman tests.

Spring Boot 3 · Spring Data JPA / Hibernate · MySQL 8 · H2 for tests · JUnit 5 · Postman

One request, four layers



Controller

HTTP only: routing, status codes and @Valid. It contains no business logic.

Service

Owns the rules and the 404 / 409 / 422 decisions. @Transactional keeps each operation atomic.

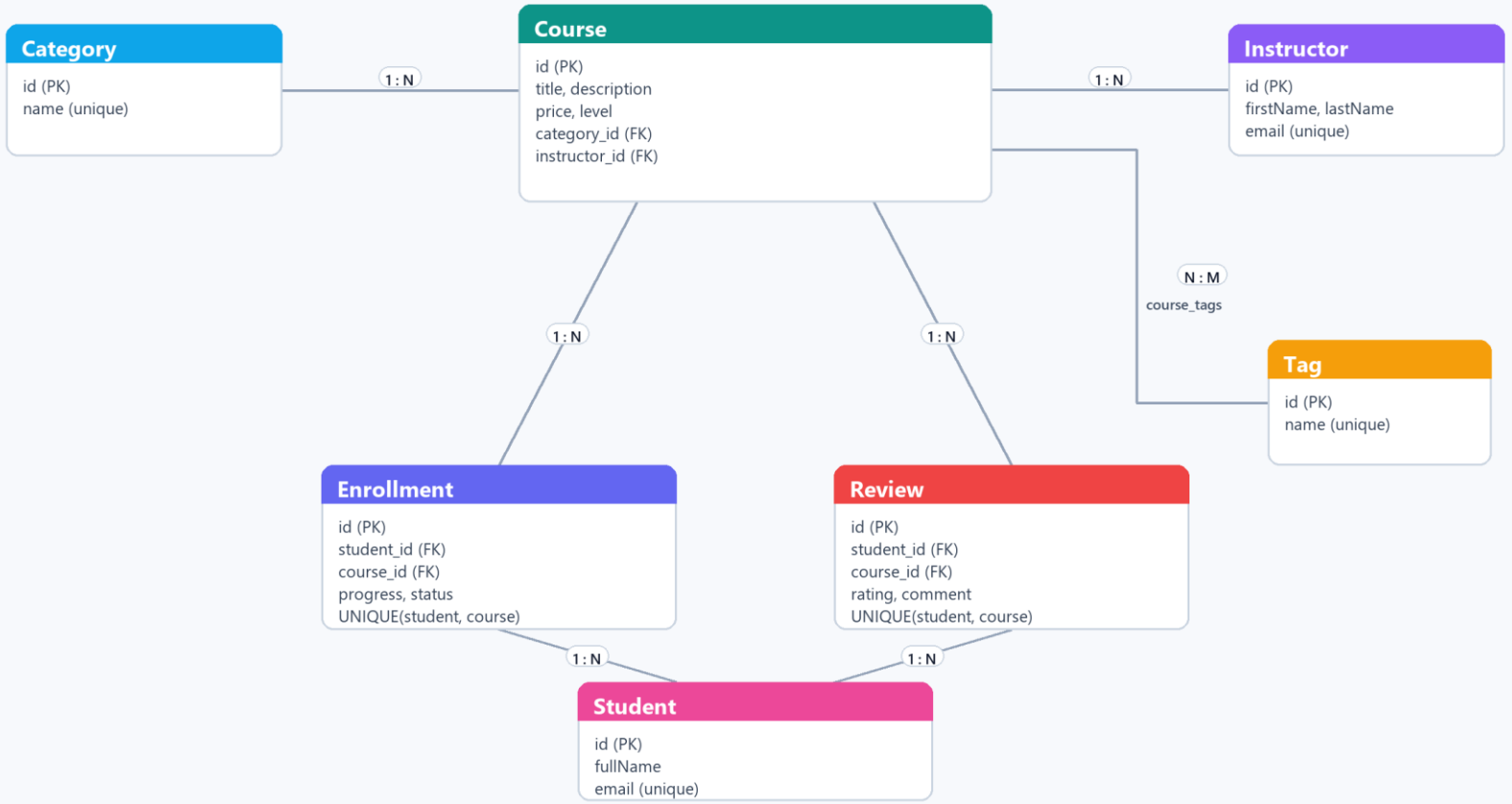
Repository

Spring Data JPA interfaces. Derived queries and five custom JPQL queries; Hibernate writes the SQL.

Model

Seven entities mapped to tables, with their relationships, constraints and validation.

Seven entities, three relationship styles



- One-to-many: Category → Course and Instructor → Course. Each course has exactly one of each.
- Many-to-many: Course ↔ Tag, stored in the course_tags join table.
- Enrollment and Review are association entities: they carry progress, status, rating and dates — data a plain join table cannot hold.
- A UNIQUE(student, course) constraint blocks double enrollment and double review.
- Deleting a referenced row is refused with 409 instead of a database error.

7 entities · 2 association entities · 1 join table

Relationships are declared once, in the entity

model/Course.java · relationships

```java
// MANY courses -> ONE category (owning side: holds the category_id foreign key)
@NotNull(message = "Category is required")
@ManyToOne(fetch = FetchType.EAGER, optional = false)
@JoinColumn(name = "category_id", nullable = false)
private Category category;
// MANY courses -> ONE instructor
@NotNull(message = "Instructor is required")
@ManyToOne(fetch = FetchType.EAGER, optional = false)
@JoinColumn(name = "instructor_id", nullable = false)
private Instructor instructor;
// MANY courses <-> MANY tags (owning side: defines the course_tags join table)
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "course_tags", joinColumns = @JoinColumn(name = "course_id"), inverseJoinColumns = @JoinColumn(name = "tag_id"))
private Set<Tag> tags = new HashSet<>();
@OneToMany(mappedBy = "course")
@JsonIgnore
private List<Enrollment> enrollments = new ArrayList<>();
```

src/main/java/com/mahmoud/coursehub/model/Course.java · lines 61–77

- @ManyToOne + @JoinColumn create category_id and instructor_id in the courses table.
- @ManyToMany + @JoinTable create course_tags(course_id, tag_id).
- mappedBy marks the inverse side; @JsonIgnore stops infinite JSON recursion.
- EAGER is used for the small reference objects that appear in every response.

TAKEAWAY Hibernate turns these declarations into joins — the API keeps returning real objects, not ids.

The service layer is where the rules live

service/EnrollmentService.java · update rules

```java
@Transactional
public Enrollment update(Long id, Enrollment data) {
    Enrollment enrollment = getById(id);
    // Business rule: a cancelled enrollment is frozen
    if (enrollment.getStatus() == EnrollmentStatus.CANCELLED) {
        throw new BusinessRuleException("A cancelled enrollment cannot be modified");
    }
    if (data.getProgress() != null) {
        if (data.getProgress() < 0 || data.getProgress() > 100) {
            throw new BusinessRuleException("Progress must be between 0 and 100");
        }
        enrollment.setProgress(data.getProgress());
    }
    if (data.getStatus() != null) {
        enrollment.setStatus(data.getStatus());
    }
    // Business rule: 100% progress means COMPLETED, and COMPLETED means 100% progress
    if (enrollment.getStatus() == EnrollmentStatus.ACTIVE && Integer.valueOf(100).equals(enrollment.getProgress())) {
        enrollment.setStatus(EnrollmentStatus.COMPLETED);
    }
    if (enrollment.getStatus() == EnrollmentStatus.COMPLETED) {
        enrollment.setProgress(100);
    }
    return enrollmentRepository.save(enrollment);
}
```

src/main/java/com/mahmoud/coursehub/service/EnrollmentService.java · lines 65–89

- One enrollment per student per course → 409 Conflict.
- A cancelled enrollment cannot be modified → 422 Unprocessable Entity.
- progress = 100 sets COMPLETED; COMPLETED forces progress = 100.
- The controller stays thin; @Transactional rolls back the whole operation on failure.

TAKEAWAY The same rule protects every client: Postman today, a future frontend tomorrow, or a script.

Five queries that answer real business questions

repository/CourseRepository.java · search

```java
// CUSTOM QUERY 1 - catalogue search: title contains a keyword AND price is under a budget
@Query("""
        SELECT c FROM Course c
        WHERE LOWER(c.title) LIKE LOWER(CONCAT('%', :keyword, '%'))
          AND c.price <= :maxPrice
        ORDER BY c.price ASC
        """)
List<Course> search(@Param("keyword") String keyword, @Param("maxPrice") BigDecimal maxPrice);
```

CourseRepository · search by title + budget

1. **Search —** title LIKE + max price
2. **Roster —** JOIN enrollments, cancelled excluded
3. **Popularity —** GROUP BY + COUNT + Pageable
4. **Revenue —** SUM of course prices
5. **Rating —** AVG of reviews

repository/EnrollmentRepository.java · popularity + revenue

```java
// CUSTOM QUERY 3 - popularity ranking: number of enrollments per course, biggest first.
// Returns one row per course: [courseId, title, enrollmentCount]. Pageable limits the rows.
@Query("""
        SELECT c.id, c.title, COUNT(e)
        FROM Enrollment e JOIN e.course c
        WHERE e.status <> :excluded
        GROUP BY c.id, c.title
        ORDER BY COUNT(e) DESC, c.title ASC
        """)
List<Object[]> findMostPopularCourses(@Param("excluded") EnrollmentStatus excluded, Pageable pageable);

// CUSTOM QUERY 4 - instructor revenue: sum of prices of the enrollments of his courses
@Query("""
        SELECT SUM(c.price)
        FROM Enrollment e JOIN e.course c
        WHERE c.instructor.id = :instructorId AND e.status <> :excluded
        """)
BigDecimal sumRevenueByInstructor(@Param("instructorId") Long instructorId,
                                  @Param("excluded") EnrollmentStatus excluded);
```

EnrollmentRepository · popularity ranking + instructor revenue

- JPQL uses entity names and fields; Hibernate translates it to the database dialect.
- Pageable limits the ranking to the top N rows.
- Popularity is returned as JSON: courseId, title, enrollments.

Bad input never reaches the database

exception/GlobalExceptionHandler.java

```java
@ExceptionHandler(BusinessRuleException.class)
public ResponseEntity<ApiError> handleBusinessRule(BusinessRuleException ex, HttpServletRequest req) {
    return build(HttpStatus.UNPROCESSABLE_ENTITY, ex.getMessage(), req, null);
}

// @Valid on a @RequestBody failed -> 400 with one message per invalid field
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex, HttpServletRequest req) {
    Map<String, String> errors = new LinkedHashMap<>();
    for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
        errors.putIfAbsent(fieldError.getField(), fieldError.getDefaultMessage());
    }
    return build(HttpStatus.BAD_REQUEST, "Validation failed", req, errors);
}
```

400 Bad Request · response body

```json
{
  "timestamp": "2026-11-09T10:15:30",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/courses",
  "fieldErrors": {
    "title": "Title must be between 3 and 150 characters",
    "price": "Price cannot be negative"
  }
}
```

`GlobalExceptionHandler` · one advice for the whole API

- Rules live on the entities: @NotBlank, @Size, @Email, @Min / @Max, @Past, @NotNull.
- @Valid on the controller parameter triggers them before the service runs.

- One ApiError shape for every failure.
- 400 validation / malformed · 404 unknown id · 409 duplicate · 422 business rule.

TAKEAWAY The messages come from the annotations; the handler only chooses the status code and the JSON shape.

Verified end to end, not just compiled

76
Postman requests

76/76
checks passed

5
JUnit tests on H2

PowerShell · scripts/validate-api.ps1 (tail)

```
PS> pwsh -File scripts/validate-api.ps1  -BaseUrl http://localhost:8081
…  66 more checks passed
PASS | DELETE student B -> 204 | expected 204 got 204
PASS | DELETE student A -> 204 | expected 204 got 204
PASS | DELETE course -> 204 | expected 204 got 204
PASS | DELETE tag 1 -> 204 | expected 204 got 204
PASS | DELETE instructor -> 204 | expected 204 got 204
PASS | DELETE category -> 204 | expected 204 got 204
PASS | Course is really gone -> 404 | expected 404 got 404
PASS | Deleting again -> 404 | expected 404 got 404
TOTAL=76 PASS=76 FAIL=0
```

Real run on the demo profile (H2) · no failures

- CRUD, the five custom queries and 25 error cases — each with status and body assertions.
- The same API runs on MySQL (default) or H2 (demo / test) by changing the profile only.
- Reproducible from the terminal: scripts/validate-api.ps1.

Demo, then questions

Live demo path

— Create category and instructor

— Create two tags and a course

— Create a student and enroll

— Update progress to 100%

— Post a review

— Show popular, rating, revenue

What it demonstrates

— 7 linked entities, 3 relationship styles

— Full CRUD on every entity

— 5 custom JPQL queries

— Validation + one error shape

— 76 passing Postman checks

Thank you

Questions?

github.com/mahmoudnajar548-hash/ds